Project Submission Description: Community Hero - Civic AI Co-Pilot

This document is the end-to-end project description for Vibe2Ship's Civic AI Co-Pilot application. It covers the problem statement, solution, architecture, features, user journeys, technical implementation, deployment, and future enhancements.

---

1. Problem Statement

Local communities struggle to report, validate, and resolve civic infrastructure issues effectively. Existing channels are often:

- fragmented across apps and government portals,
- slow to provide feedback,
- lacking trust and verification,
- without clear tracking of issue resolution,
- difficult for citizens to know which department owns the work.

Selected Challenge

Community Hero addresses the hyperlocal infrastructure problem by enabling citizens to report hazards such as:

- road potholes,
- overflowing garbage,
- damaged streetlights,
- water leaks and blocked drains.

The goal is to unify reporting, verification, tracking, and resolution into one AI-powered civic co-pilot.

---

2. Solution Summary

Community Hero is a full-stack civic collaboration platform that empowers citizens and field workers with AI-assisted workflows.
It includes:

- multimodal **image/video reporting**,
- AI-driven **issue categorization and severity scoring**,
- **geo-location mapping** with interactive coordinate selection,
- **community verification** through upvotes and flags,
- **field worker task assignment** and real-time status tracking,
- **AI resolution proof auditing**,
- gamified community points and badges,
- AI-powered **predictive infrastructure insights**.

This solution bridges citizen reports and municipal response using a reactive frontend and a resilient backend.

---

3. End-to-End Architecture

Frontend
Built as a React SPA with Vite, the frontend provides:
- a responsive dark-themed dashboard,
- an AI evidence upload console,
- issue list cards with expandable timelines,
- live map location selection,
- role switching between **Citizen** and **Field Worker**,
- gamification and leaderboard.

Backend
FastAPI powers the backend with:
- REST endpoints for issue CRUD and verification,
- image upload analysis and resolution proof handling,
- a repository-pattern database layer,
- flexible storage using MongoDB with an automatic SQLite fallback.

AI Integration
Google Gemini is used for:
- multimodal classification of uploaded civic evidence,
- automated title/description generation,
- department routing and dispatch order drafting,
- completion proof audit and verification feedback.

Data Persistence
Data flows through:
- persistent database tables for issues and users,
- local cache fallback via `localStorage` for offline-friendly operation,
- seeded initial issues for demo readiness.

---

4. Core Features

4.1 AI-Powered Issue Reporting
- Citizens upload a photo or video of a hazard.
- The system analyzes the media using Gemini.
- Results auto-fill the report form with:

- category,
- severity,
- title,
- description,
- recommended department,
- dispatch order,

- priority score,
- estimated completion time.

4.2 Geo-Location and Mapping
• Reporters receive a default geotag from the analysis flow.
• Leaflet map allows precise coordinate adjustments.
• Issues are displayed with map markers and location summaries.

4.3 Community Verification and Moderation
• Citizens can verify issues using upvotes.
• They can flag questionable reports for review.
• Verified issues strengthen trust and trigger progress tracking.

4.4 Role-Based Workflow
• **Citizen Role**: report hazards, verify issues, track resolution status.
• **Field Worker Role**: claim tasks, advance work status, upload completion proof.

4.5 AI Resolution Proof Audit
• Workers upload a photo showing completed repair.
• The backend routes the proof to Gemini for verification.
• If verified, the issue status updates to **Resolved**.
• If rejected, the issue returns to **In Progress** with audit feedback.

4.6 Impact Dashboard and Predictive Insights
• The dashboard displays live metrics:
- active issue count,
- resolved issue count,
- high-priority issues,
- community impact points.
• The AI insights sidebar presents predictive alerts for infrastructure risks.

4.7 Gamification and Community Engagement
• Users earn points for reporting and verifying issues.
• Leaderboard tracks top contributors.
• Badges reward engagement and verification milestones.

---

5. User Journeys

Journey 1: Citizen Reporting a New Issue
1. Switch to **Citizen** role.
2. Upload an image or video of a civic hazard.
3. Watch the AI console log analysis and classification.

4. Review and refine the auto-filled report.
5. Place or adjust the map pin for exact location.
6. Submit the issue and earn community points.

Journey 2: Community Verification
1. Browse reported issues in the dashboard.
2. Read issue cards and inspection logs.
3. Upvote valid reports to verify them.
4. Flag incorrect or duplicate reports.
5. The system updates the issue status and trust signals.

Journey 3: Field Worker Repair and Audit
1. Switch to **Field Worker** mode.
2. Claim an assignment from the active issue list.
3. Advance the workflow to **In Progress**.
4. Upload a repair proof image after work completion.
5. Trigger the AI audit and receive verification feedback.
6. View the resolved issue with a certified proof badge.

---

6. Technical Implementation Details

Backend Components
- `backend/main.py`: initializes FastAPI and routes.
- `backend/database/connection.py`: abstracts MongoDB/SQLite connectivity.
- `backend/database/setup.py`: defines collections, auto-migration, and seed data.
- `backend/models/schemas.py`: validates requests/responses with Pydantic.
- `backend/repositories/issue_repository.py`: manages ticket lifecycle and history.
- `backend/services/gemini_service.py`: builds prompts and handles Gemini interactions.
- `backend/routers/analysis.py`: processes file uploads for AI pre-fill.
- `backend/routers/issues.py`: implements issue management and proof validation.

Frontend Components
- `frontend/src/components/ReportIssue.jsx`: AI upload console, review form, and map editor.
- `frontend/src/components/Dashboard.jsx`: issue feed, metrics, role-specific actions, and predictive insights.
- `frontend/src/context/IssueContext.jsx`: shared state, backend sync, local fallback, and issue management.
- `frontend/src/utils/gemini.js`: Gemini demand generation and

fallback simulation.
• `frontend/vite.config.js`: development proxy to backend API.

AI Prompt Strategy

• Gemini is prompted to return structured JSON with keys for category, severity, title, description, department, dispatch order, priority score, and estimated completion.
• The application handles malformed responses gracefully by falling back to simulated defaults.
• Visual audit proof uses text comparisons and verification logging.

Resilience and Fallbacks

• If the FastAPI backend or Gemini API is unavailable, the frontend continues to operate in local simulation mode.
• `localStorage` persists issue state, user profile, and API key preferences.
• The backend supports SQLite auto-migration for environments without MongoDB.

---

7. Installation & Deployment

Prerequisites

• Node.js v18+
• npm
• Python 3.10+
• pip

Setup

1. Install frontend and backend dependencies:

```bash
npm run install:all
```

2. Create `.env` from `.env.example` and configure:

```env
VITE_GEMINI_API_KEY=your_gemini_api_key
MONGODB_URI=your_mongodb_uri
MONGODB_DB_NAME=vibe2ship
```

3. Start the application:

```bash
npm run dev
```

Production Build
To build the frontend for production:

```bash
npm run build --prefix frontend
```

---

8. Deployment Considerations

- Use a managed MongoDB cluster or Atlas database for persistence.
- Host the backend on services like **Railway**, **Fly.io**, or **Heroku**.
- Serve the frontend from **Vercel**, **Netlify**, or **Firebase Hosting**.
- Ensure the Gemini API key is stored securely in environment variables.

---

9. Project Impact

This solution delivers:

- faster and more reliable civic issue reporting,
- trust through community verification,
- transparency with ticket history and audit proof,
- operational efficiency for field crews,
- predictive guidance to prevent future hazards,
- community engagement through gamified points and badges.

---

10. Future Enhancements

Potential next steps include:

- real-time WebSocket updates for immediate issue status changes,
- mobile-friendly upload flow with camera capture,
- advanced duplicate detection using spatial clustering,
- richer AI proof validation via object detection,
- multilingual citizen reporting,
- exportable municipal performance dashboards.

---

11. Repo Structure Summary

```text
Vibe2Ship/
■■■ backend/
■   ■■■ database/
■   ■■■ models/
```

■ ■■■ repositories/
■ ■■■ routers/
■ ■■■ services/
■ ■■■ static/
■ ■■■ main.py
■ ■■■ requirements.txt
■■■ frontend/
■ ■■■ src/
■ ■ ■■■ components/
■ ■ ■■■ context/
■ ■ ■■■ utils/
■ ■■■ package.json
■ ■■■ vite.config.js
■■■ .env.example
■■■ package.json
■■■ README.md
■■■ PROJECT_DESCRIPTION.md

```

---

## 12. Conclusion

Community Hero is a complete end-to-end civic AI co-pilot. It guides citizens from hazard detection to municipal verification and resolution, while giving municipal workers a practical workflow to claim, execute, and certify repairs. The project demonstrates a strong fusion of AI, geospatial mapping, verification workflows, and community-driven civic engagement.
```